

Tensorization Techniques for Neural Networks: A Comprehensive Overview by Model Type (2015–2026)

TL;DR

- **Tensorization — reshaping weight matrices/kernels into higher-order tensors and approximating them with low-rank tensor-network decompositions (CP, Tucker, TT, TR, HT, BTD, MPO) — is now a mature compression and acceleration tool across all major architectures, and the 2023–2026 frontier has shifted decisively from CNNs toward LLMs, where tensor methods deliver up to 93–95% size reduction (CompactifAI) and parameter-efficient fine-tuning with up to $\sim 1560\times$ fewer trainable parameters than full tuning (TT-LoRA).**
- **The technique-to-architecture mapping is now well established:** Tucker/CP for convolutional kernels (CNNs), TT/TR for fully-connected and embedding layers (MLPs, RNNs), MPO/Block-Term/Tucker for attention and feed-forward layers (transformers), and SVD + tensor-train LoRA variants for LLMs — increasingly combined with quantization and pruning for multiplicative gains.
- **For Ema Puljak's deck, the highest-value extensions are:** (1) the LLM tensor-network line (CompactifAI/MPO, SVD-LLM/ASVD, LoTR/TT-LoRA/TensLoRA), (2) automatic/hardware-aware rank search (HEAT, AutoRank, TR-VB), and (3) the convergence of tensor methods with quantization (Quantization-Aware Factorization, CompactifAI+4-bit) — all of which post-date the seminal Kim 2016 / Lebedev / Novikov work the deck centers on.

Key Findings

1. **Tensorization is complementary to quantization, pruning, and distillation, and the best results combine them.** CompactifAI + 4-bit quantization reduces LLaMA-2 7B memory by 93%; the position paper by Hamreras, Singh & Orús (arXiv:2505.20132) argues tensorization targets the model's "correlation space" rather than neuron count, a fundamentally different and arguably more principled axis of compression.
2. **CNNs:** The canonical recipes are CP decomposition of the 4D conv kernel into four small convolutions (Lebedev et al. 2015) and Tucker-2 decomposition with VBMF rank selection + whole-network fine-tuning (Kim et al. 2016 — the deck's anchor). Newer work pushes to Hierarchical Tucker-2, Tensor Ring with Bayesian rank selection, stable/sensitivity-based CP, and ADMM-trained Tucker formats that beat FLOPs-based rank selection.
3. **MLPs / fully-connected:** Novikov et al.'s TT-layer (NeurIPS 2015) achieves up to $200,000\times$ compression of a single VGG FC matrix ($7\times$ whole-network). TT/TR tensorized embeddings are now standard for NLP vocabulary matrices.
4. **Transformers:** Block-Term Decomposition multi-linear attention (Ma et al. 2019), MPO decomposition of BERT/ALBERT layers (Liu et al., ACL 2021), and hardware-aware

automatic tensor decomposition (HEAT, NVIDIA 2022) that achieves $5.7\times$ lower energy-delay product on BERT with $<1.1\%$ accuracy loss.

5. **LLMs (the frontier):** CompactifAI (MPO/tensor networks), SVD-family weight compression (FWSVD $\rightarrow$ ASVD $\rightarrow$ SVD-LLM $\rightarrow$ SVD-LLM V2/AdaSVD/ARA), and tensorized LoRA variants (LoTR, TT-LoRA, LoRETTA, TensLoRA, TensorGuide). These are the most active 2024–2026 areas.
6. **Rank selection** has evolved from VBMF (Kim 2016) and sensitivity analysis to NAS-style SuperNets (HEAT), genetic search (AutoRank), Bayesian/variational methods (TR-VB), and differentiable L1 rank learning (TensorLy-Torch).
7. **Software** is mature: TensorLy + TensorLy-Torch, TedNet, t3nsor/TensorNet, and the awesome-tensorial-neural-networks survey repo cover all decomposition families.

Details

1. Decomposition families (the mathematical toolkit)

A tensor is reshaped/decomposed into a network of smaller "core" tensors contracted over internal "bond" indices. The main families, in rough order of structural complexity:

- **SVD / low-rank matrix factorization** — the order-2 special case; $W \approx U\Sigma V^T$ truncated to rank r . Tucker-1 $\equiv$ SVD. The basis for all LLM SVD methods.
- **CP (CANDECOMP/PARAFAC)** — sum of R rank-one tensors; most aggressive parameter reduction but numerically unstable (degeneracy) and hard rank selection. Lebedev 2015 for conv; Phan et al. 2020 (ECCV) "Stable Low-Rank Tensor Decomposition" fixed the instability via a sensitivity-aware approach.
- **Tucker (and Tucker-2/Tucker-1)** — a core tensor multiplied by factor matrices along each mode; more stable than CP, "strictly superior in almost every way" per practitioners. Tucker-2 on the input/output channel modes is the workhorse for conv layers.
- **Tensor-Train (TT) / Matrix Product Operator (MPO)** — a linear chain of 3-index cores; parameter count scales as $O(dnr^2)$, avoiding Tucker's exponential core. TT (Oseledets 2011) is identical to the MPO of quantum many-body physics. Best for high-order reshapes of FC/embedding matrices.
- **Tensor Ring (TR)** — TT with periodic boundary (the chain closes into a ring); more expressive/flexible rank allocation than TT. Wang et al. 2018 "Wide Compression: Tensor Ring Nets."
- **Hierarchical Tucker (HT)** — binary-tree of cores; Yin et al. (2020/2021) showed HT-LSTM beats TT/TR/BT-LSTM in both compression and accuracy. ([arxiv](#))
- **Block-Term Decomposition (BTD)** — a sum of several Tucker blocks; interpolates between CP and Tucker. Foundation of the Tensorized Transformer.
- **Kronecker products** — $W \approx A \otimes B$; used in RNN compression (KCP) and structured-matrix LLM compression.
- **Other TN structures (MERA, PEPS)** — drawn from physics; mostly still exploratory for ML, flagged in the 2505.20132 position paper as future directions.

Complexity note: a full $M \times N$ FC layer costs MN parameters; the TT-layer's limiting factor is $\max\{M, N\}$, and TT scales quadratically in rank ($O(dnr^2)$) — this is the $O(n^2)$ -type analysis the deck references.

2. CONVOLUTIONAL NETWORKS (CNNs)

The 4D kernel. A conv layer's kernel is a tensor $K \in \mathbb{R}^{(T \times S \times D \times D)}$ (output channels $\times$ input channels $\times$ spatial $\times$ spatial). Decomposition replaces one expensive convolution with a sequence of cheaper ones.

- **CP decomposition (Lebedev, Ganin, Rakhuba, Oseledets, Lempitsky, ICLR 2015, arXiv:1412.6553).** Approximates the kernel as a sum of rank-one tensors $\rightarrow$ four small convolutions (1×1 , two separable spatial, 1×1). Reported: AlexNet 2nd conv layer **4 $\times$ speedup at $\sim 1\%$ top-5 error increase**; a 36-class character CNN got **8.5 $\times$ whole-network CPU speedup with accuracy dropping only 1% (91% $\rightarrow$ 90%)**. Fine-tuning the whole network with backprop recovers the large CP approximation error.
- **Tucker-2 one-shot whole-network compression (Kim, Park, Yoo, Choi, Yang, Shin, 2016, arXiv:1511.06530 — the deck's anchor).** Three steps: (1) **rank selection via global analytic VBMF** on mode-3/mode-4 matricizations of each kernel; (2) **Tucker decomposition** on each kernel; (3) **fine-tune the entire network**. Tucker-2 is applied from the second conv layer through the first FC layers; Tucker-1 ($\equiv$ SVD) to the others. Tested on AlexNet, VGG-S, GoogLeNet, VGG-16 on a smartphone, achieving significant reductions in model size, runtime, and energy at small accuracy cost. (arXiv + 2)
- **Spatial/separable convolutions as decomposition.** Depthwise-separable convolution (MobileNet) is mathematically a hand-designed low-rank factorization: a depthwise spatial conv followed by a 1×1 pointwise conv — i.e., the architecture is a baked-in low-rank structure. This connects the "compact-design" and "low-rank-decomposition" literatures; a CP-decomposed conv yields exactly a separable convolution (the basis of TensorLy-Torch's FactorizedConv).
- **Recent CNN advances (2021–2025):**
 - **Hierarchical Tucker-2 (Gabor & Zdunek, Knowledge-Based Systems 2022)** — represents each conv layer with four smaller sublayers; substantial parameter + FLOPs reduction at marginal accuracy drop; introduces "Kruskal convolution" notation. (ScienceDirect)
 - **Funnel regularization (Chu & Lee, arXiv:2112.03690)** — global rank selection via a "funnel" function; ResNet-18 on ImageNet **>2 $\times$ GMAC speedup at only 0.7% top-1 drop**.
 - **Tensor Ring + Variational Bayes (TR-VB, Neural Processing Letters 2024)** — uses the Global Analytic Solution of EVBMF for per-layer TR rank; evaluated on ResNet-20/32/56, CIFAR-10/100.
 - **Reduced-Storage Direct TR (RSDTR, arXiv:2405.10802)** — selects the minimal-storage TR representation; compresses parameters *and* FLOPs simultaneously (unlike

earlier TR that traded FLOPs for params), and can fine-tune from decomposed factors instead of training from scratch.

- **Stable/sensitivity-based CP** (MDPI *Applied Sciences* 2024; Phan et al. ECCV 2020) — fixes CP's degeneracy/instability problem.
- **Distribution-Aware Tensor Decomposition** (arXiv:2511.04494, late 2025) — minimizes a functional/data-aware norm instead of the Frobenius norm, on ResNet-18/50, GoogLeNet, ImageNet, FGVC-Aircraft. (arxiv)
- **ADMM-based Tucker/TT** (arXiv:2107.12422) — on CIFAR-100, ADMM-trained TT-format ResNet-20/32 achieved **+1.96%/+2.21% top-1 over the original** at $2.3\times/2.4\times$ compression; ResNet-18 ImageNet $2.47\times$ FLOPs reduction without accuracy loss.
- **End-to-end Tucker + GPU codesign** — ADMM-trained Tucker models with optimized kernels reached up to $2.21\times$ over cuDNN, $1.12\times$ over TVM, $3.27\times$ over original, with ranks chosen by *measured inference time* rather than FLOPs.
- **LoRA-Edge** (arXiv:2511.03765, Nov 2025) — TT-SVD applied to pretrained conv layers for on-device fine-tuning (Human Activity Recognition): accuracy within 4.7% of full fine-tuning while updating $\leq 1.49\%$ of parameters; $1.4\text{--}3.8\times$ faster convergence on Jetson Orin Nano. (arXiv)

Trade-off summary (CNNs): CP gives highest compression but lowest stability; Tucker is the robust default; TT/TR/HT give very high compression on FC-heavy or recurrent nets but can increase FLOPs unless rank/reshape is co-optimized (RSDTR addresses this). Whole-network fine-tuning after decomposition is essential.

3. DENSE / FULLY-CONNECTED LAYERS (MLPs)

- **TT-layer / TensorNet** (Novikov, Podoprikin, Osokin, Vetrov, NeurIPS 2015, arXiv:1509.06569 — a deck-adjacent anchor). Reshapes the dense FC weight matrix into a high-order tensor and stores it in TT format. Verbatim result: "**compression factor of the dense weight matrix of a fully-connected layer up to 200000 times leading to the compression factor of the whole network up to 7 times**" (concretely $\sim 194,622\times$ on a VGG-19 25088×4096 FC matrix $\rightarrow$ 528 params), preserving accuracy on MNIST/CIFAR-10/ImageNet. The limiting size becomes $\max\{M,N\}$ instead of MN. Implementations: TensorNet (MATLAB/Theano), TensorNet-TF. (arXiv + 2)
- **Ultimate Tensorization** (Garipov, Podoprikin, Novikov, Vetrov, 2016) — extends TT to convolutional and FC layers alike ("compressing convolutional and FC layers alike"). (GitHub)
- **Tensorized embeddings (TT/TR)**. NLP embedding/lookup matrices (huge vocab $\times$ dim) are reshaped and TT/TR-decomposed; slightly *better* test accuracy is often observed, suggesting low-rank tensor structure acts as regularization.
- **MLP-Mixer / fully-tensorized vision MLPs** — covered by TensorLy-Torch factorized linear layers and the TNN position paper's "stack view."

4. TRANSFORMERS

- **Tensorized Transformer / Multi-linear attention** (Ma, Zhang, Zhang, Duan, Hou, Zhou, Song, NeurIPS 2019, arXiv:1906.09777). Reconstructs Scaled Dot-Product attention as a **single-block Tucker decomposition (3-order tensor)**, then compresses multi-head attention via **Block-Term Decomposition** sharing factor matrices across blocks. On PTB, WikiText-103, One-Billion, and WMT-2016 En-De, it **both compresses parameters and improves performance** over the standard Transformer.
- **MPO decomposition of PLMs** (Liu, Gao, Zhao, Xie, Lu & Wen, ACL 2021, arXiv:2106.02205 — "Enabling Lightweight Fine-tuning..."). Decomposes each linear weight into **central tensors (core information) + auxiliary tensors (small)**; fine-tunes only auxiliary tensors → verbatim "**91% reduction on average**" in fine-tuning parameters on ALBERT/BERT/DistilBERT/MobileBERT (GLUE). Introduces "dimension squeezing" to control reconstruction-error accumulation across stacked layers. (arxiv + 2)
- **HEAT — Hardware-Efficient Automatic Tensor Decomposition** (Gu, Keller, Kossaifi, Anandkumar, Khailany & Pan; NVIDIA/UT Austin/Caltech, 2022, arXiv:2211.16749 — the deck's "hardware-aware/automatic" line). Three-level optimization: (1) simulate each tensorization **shape** candidate on the target accelerator, build a **hardware cost table**, pick a **Pareto-optimal shape**; (2) a weight-sharing **Rank SuperNet** explores the exponential per-tensor rank space in one shot; (3) **knowledge-distillation retraining** of the searched factorized model. Verbatim result: hardware-aware factorized BERT variants "**reduce the energy-delay product by 5.7x with less than 1.1% accuracy loss,**" surpassing hand-tuned/heuristic baselines by "25%-30% lower EDP and 1-3% accuracy improvement on the SQuAD-v1.1 and SST-2 datasets." Includes Einsum-fusion and contraction-path optimization to bridge theoretical and real efficiency. (arXiv + 2)
- **AutoRank (ICCB 2024, ACM)** — genetic-algorithm automatic tensor-rank search + fine-tuning on AlexNet/VGG11_bn, reducing GPU memory and power.
- **Hypoformer (Hybrid TT decomposition)** and **Tuformer (Tucker-format data-driven transformer)** — extend tensorization to edge-friendly NMT and generalization-improving attention, respectively.
- **Tensorized Attention Model (TAM, ICLR 2024 submission)** — Tucker-based attention across multiple object types (Reddit, TweetQA), going beyond pairwise relationships.

5. LARGE LANGUAGE MODELS (LLMs) — the 2023-2026 frontier

(a) **Tensor-network (MPO) compression — CompactifAI (Multiverse Computing; Tomut, Jahromi, Singh, ..., Orús et al., arXiv:2401.14109)**. Replaces the weight matrices of the **Self-Attention (SA)** and **MLP** layers of each LLaMA decoder block with **Matrix Product Operators of bond dimension χ** , computed via sequential SVDs retaining the largest χ singular values. Key finding: early layers/blocks are sensitive to truncation; later layers can be compressed to ~10% of size. Headline LLaMA-2 7B result (Table I/II, verbatim): combining CompactifAI with quantization "**reduce[s] a 93% the memory size of LLaMA-2 7B, reducing also 70% the number of parameters, accelerating 50% the training and 25% the inference times of the model, and just with a small accuracy drop of 2% - 3%.**" Concretely: 27.1 GB / 7B / float-32 → 2.1 GB / 2.1B

parameters (mixed float-16 tensorized + int-4 non-tensorized). Retraining ("healing") used **less than one epoch** on generic chat datasets (Ultrachat, Alpaca, OpenHermes) on 8× NVIDIA A10g GPUs. Commercially, Multiverse Computing raised a **€189 million (~\$215 million) Series B led by Bullhound Capital in June 2025** (TechCrunch; investors include HP Tech Ventures, Forgepoint, Toshiba), and released "Llama 3.1-8B Slim / 3.3-70B Slim" — 80% compressed, 60% fewer parameters, 84% greater energy efficiency, 40% faster inference, ~50% cost reduction (the press release claims compression "by up to 95% with only 2-3% precision loss"; these are vendor figures). An independent arXiv:2507.08836 study (SopraSteria) broadly corroborates the efficiency + accuracy retention on Llama 3.1 8B. The position paper arXiv:2505.20132 frames the approach as targeting "correlation space" rather than neuron count. (arxiv + 8)

(b) SVD-family weight compression (training-free or light-retraining):

- **FWSVD (Hsu et al., 2022)** — Fisher-information-weighted SVD; weights important parameters, uses retraining.
- **ASVD (Yuan et al., 2023, arXiv:2312.05821)** — **Activation-aware SVD**: scales weight matrix by a diagonal matrix normalizing input-channel influence; first to compress LLM weights via decomposition; also compresses KV-cache by decomposing K/V projections (up to 50% KV-cache memory reduction); Sensitivity-based Truncation Rank Searching sets per-layer ratios.
- **SVD-LLM (Wang et al., ICLR 2025, arXiv:2403.07378)** — **truncation-aware** data whitening establishing a direct singular-value↔truncation-loss relationship; outperforms FWSVD/ASVD especially at high compression; orthogonal to quantization/pruning. A LLaMA-3B compressed from LLaMA-7B by SVD-LLM beats from-scratch StableLM-3B on accuracy, throughput, and memory.
- **SVD-LLM V2 (arXiv:2503.12340), AdaSVD (arXiv:2502.01403), ARA (arXiv:2510.19389)** — adaptive/optimized rank allocation across layers; Dobi-SVD and SoLA combine SVD with quantization / activation-channel preservation.
- **Structured matrices + Kronecker (arXiv:2506.02818), Low-Rank Correction for Quantized LLMs (arXiv:2412.07902)** — Kronecker/structured factorization and low-rank correction to mitigate quantization outliers; at ranks ~30% of the weight matrix the accuracy gap to FP closes (Llama-2/3, Phi-3, Mixtral).
- **Palu (ICLR 2025)** — KV-cache compression via low-rank projection of K/V + Hadamard transform to remove outliers.

(c) Tensorized parameter-efficient fine-tuning (tensor LoRA family):

- **LoTR (Bershatsky, Cherniuk et al., 2024, arXiv:2402.01376)** — represents the gradient update across *multiple layers jointly* as a tensor decomposition (each layer's adapter = product of three matrices, with shared left/right factors across layers). Parameter count $O(Lr^2 + dr)$ decouples depth L from width d — better efficiency than LoRA especially for deep models.

- **LoRETTA** (Yang, Zhou, Wong, Zhang, NAACL 2024, arXiv:2402.11417) — tensor-train adapters (LoRETTA_adp) and TT reparameterization (LoRETTA_rep). Verbatim: "**up to 100× fewer parameters on the LLaMA-2-7B models**" vs widely-used PEFT methods. LoRETTA_rep needs <1MB / 0.51M params; on DeBERTa 0.05–0.10M params at ~85 GLUE avg. On LLaMA-2 70B, LoRETTA_adp achieves "a reduction of nearly 12 million trainable parameters ... with over 1% accuracy improvement" vs LoRA. Storage cut by 57.4× (rep) and 9.8× (adp) vs LoRA on LLaMA-2. (arXiv + 2)
- **TT-LoRA** (Anjum, Eren, Boureima, Alexandrov, Bhattarai; LANL; IEEE ICMLA 2024, arXiv:2408.01008) — drops adapters entirely, decomposing ΔW (on W_q, W_v) directly into TT cores. Illustrative example: a $768 \times 2304 \Delta W$ (1,769,472 params) $\rightarrow$ 1,135 params at TT-rank 5, a **1560× compression ratio**. vs LoRA: **41.9× fewer params on LLaMA-2 7B**, **31.5× on RoBERTa**, **170.5× on LLaMA-3 8B**, matching or beating accuracy (LLaMA-2 7B avg 80.19, best of all methods, at 0.1M params; LLaMA-2 7B TT-LoRA needs only ~195 KB of adapter storage). (arxiv + 5)
- **TensLoRA** (Marmoret et al., 2025, arXiv:2509.19391) — unifying framework aggregating LoRA updates into higher-order tensors (Att/QKV/Depth), generalizing FacT, LoTR, LoRTA, CaRA with mode-specific compression rates. (ResearchGate)
- **TensorGuide** (arXiv:2506.16456) — generates two correlated LoRA matrices through a unified TT structure driven by Gaussian noise; outperforms LoRA and TT-LoRA on GPT-2 with NTK-based theoretical justification.
- **TT-LoRA MoE** (arXiv:2504.21190) — combines TT-LoRA experts with sparse MoE routing for multi-task scaling.

6. OTHER ARCHITECTURES

- **RNNs/LSTMs** — the highest-compression success stories: TT-RNN (Yang, Krompass, Tresp, ICML 2017) for video; TR-LSTM, BT-LSTM, and **HT-LSTM** (Yin et al. 2020) which beats them all; **KCP-RNN (Kronecker CP)** reporting up to ~278,000× weight compression on video/speech datasets with comparable accuracy. CP/Tucker/TT-GRU comparisons (Tjandra et al. 2018) found TT-GRU best at equal parameter budgets. Tensor decomposition is noted as "uniquely" effective for RNNs because their recurrent weight reuse amplifies the compression benefit.
- **Vision Transformers** — FacT (Jie & Deng, AAAI 2023, arXiv:2212.03145): tensorizes ViT weights into a single 3D tensor, decomposes the *increments* with TT (FacT-TT) or Tucker (FacT-TK); on VTAB-1K matches state-of-the-art NOAH at **5× fewer parameters**, with a tiny version using only 8K (0.01%) trainable params. EFFT (arXiv:2311.06749) extends it to 75.9% VTAB-1K avg at 0.28% params. (arXiv + 3)
- **Graph neural networks / diffusion models** — tensorization is emerging but comparatively thin; activation-map compression via HOSVD (NeurIPS 2024, "Activation Map Compression through Tensor Decomposition") targets on-device backprop memory rather than weights and applies across architectures, with theoretical convergence guarantees.

7. CROSS-CUTTING TOPICS

Rank selection / determination:

- **VBMF / global analytic VBMF** (Kim 2016) — closed-form, the classic.
- **Sensitivity-based** (ASVD, stable CP) — per-layer ratios by output sensitivity.
- **Bayesian / Variational** (TR-VB via EVBMF; Bayesian TN structure search, *Neural Networks* 2024).
- **NAS / SuperNet** (HEAT Rank SuperNet; AutoRank genetic algorithm).
- **Differentiable / L1 rank learning** (TensorLy-Torch L1 regularization + tensor dropout).
- **Singular-value-thresholding (SVT)** for partial Tucker.

Training strategies:

- **Decompose-then-finetune** (Lebedev, Kim — the dominant "direct" method; factors initialize new weights).
- **Train-from-scratch in factorized form** (Novikov TT-layer, TR-Nets — "tensorization" method; hand-designed tensorized layers).
- **Progressive / dimension-squeezing** (MPO PLM) to control accumulated reconstruction error in deep stacks.
- **ADMM-constrained training** — augmented Lagrangian $L_{\text{ADMM}}(W, Z, U) = L(W) + g(Z) + (\rho/2)\|W - Z + U\|^2$ imposes low-TT-rank/low-Tucker-rank structure during training, then decomposes and fine-tunes; also used for joint pruning and **Quantization-Aware Factorization** (arXiv:2308.04595) that co-optimizes low-rank factors and quantization grids.

Relationship to other compression methods: Tensorization is orthogonal and combinable — quantize a tensorized model (CompactifAI+4-bit, Dobi-SVD), prune+factorize (ADMM joint), or distill a dense teacher into a tensorized student (HEAT). The 2505.20132 position paper argues combinations consistently outperform any single method.

Hardware / efficiency implications: Theoretical FLOPs/parameter savings do NOT automatically translate to latency — naive TT/TR can *increase* FLOPs and many decomposed ops are memory-bound or poorly mapped to GPUs. This motivated hardware-aware lines: HEAT (energy-delay product, Einsum fusion, contraction-path optimization), measured-latency rank selection, and edge results (LoRA-Edge on Jetson, Kim 2016 on smartphones). SVD methods are praised as more hardware-friendly than quantization (no custom kernels needed) and effective because LLM inference is memory-bound. (arXiv)

Software libraries:

- **TensorLy** (NumPy/PyTorch/TF/JAX/CuPy/Paddle backends) — core decompositions (CP, Tucker, TT, etc.).

- **TensorLy-Torch (tltorch)** — out-of-the-box FactorizedConv (CP=separable, Tucker, TT conv), tensor regression/contraction layers, tensorized linear layers, tensor dropout, end-to-end L1 rank learning, factorized init from pretrained.
- **TedNet** (PyTorch, MIT license) — CP/Tucker/TT/TR/BT/HT layers and TD-ResNets/TD-RNNs.
- **TensorNet / TensorNet-TF / t3nsor** — original Novikov TT-layer code.
- **awesome-tensorial-neural-networks** — curated survey repository (living taxonomy of tensorial NNs).

Theoretical foundations: TNNs expose new latent "bond" degrees of freedom; the geometry of the TN reflects weight correlation structure (interpretability angle). TT scales as $O(d_{nr}^2)$ vs the MN of dense FC. Tucker suffers curse-of-dimensionality in its core; CP has limited expressive power and ill-posed rank. HT/TR/BTD trade structure for expressiveness. Tensor networks originate in quantum many-body physics, where they represent wavefunctions whose size grows exponentially with system size using parameters that scale only linearly. (ScienceDirect)

Open challenges / future directions (per arXiv:2505.20132 and others): (1)

trainability/initialization of decomposed factors (numerical instability, esp. CP); (2) translating theoretical savings into real hardware speedups; (3) automatic, principled rank/shape/topology selection; (4) "fully tensorized" networks (weights + activations + activation functions + normalization all as TNs); (5) exploiting bond indices for mechanistic interpretability; (6) scaling tensor methods to frontier (100B+) LLMs; (7) exotic TN topologies (MERA, PEPS) for ML.

Recommendations

For extending the deck, in priority order:

1. **Add a dedicated LLM section** — this is where the field has moved and where the deck is currently thinnest. Lead with CompactifAI/MPO (the tensor-network story, with the LLaMA-2 7B 93% memory / 70% parameter figures and the Multiverse €189M Series B commercial traction), then SVD-LLM/ASVD (the practical training-free SVD line), then the tensorized-LoRA family (LoTR → LoRETTA → TT-LoRA → TensLoRA) for PEFT.
Benchmark that would change emphasis: if a tensor-network method demonstrates near-lossless compression of a >70B reasoning model on hard benchmarks (GSM8K/MMLU) in a peer-reviewed venue, elevate it to the headline.
2. **Frame the automatic/hardware-aware line as the natural successor to Kim 2016's VBMF.** Use HEAT (NVIDIA) as the flagship example of the three-level shape→rank→retrain pipeline the deck already gestures at, and contrast FLOPs-based vs measured-latency rank selection.
3. **Add a "tensorization × quantization × pruning" convergence slide** — Quantization-Aware Factorization, CompactifAI+4-bit, ADMM joint pruning. This reframes tensorization as one composable axis rather than a standalone method.

4. **Keep CP/Tucker/TT/TR/HT/BTD as a unified taxonomy slide** organized by structural complexity, then map each to its "home" architecture (Tucker→CNN, TT→FC/embedding, MPO/BTD→transformer attention, HT/KCP→RNN). Cite the awesome-tensorial-neural-networks repo as the living taxonomy.
5. **Demo with TensorLy-Torch** (FactorizedConv, factorized linear, L1 rank learning) for a live, reproducible example, and reference TedNet for the broader decomposition zoo.

Thresholds for method choice (practical guidance to include):

- Need robust, low-risk CNN compression → **Tucker-2 + VBMF + fine-tune** (Kim 2016).
- Need maximal compression and can tolerate instability/fine-tuning effort → **CP (stable variant)** or **TT/TR**.
- Compressing FC/embedding → **TT-layer**.
- Compressing transformer attention/FFN with a hardware target → **HEAT / MPO**.
- Compressing a deployed LLM → **SVD-LLM (training-free)** or **CompactifAI (with healing)**; combine with 4-bit quantization.
- Fine-tuning an LLM cheaply → **TT-LoRA / LoRETTA** (100×+ fewer params than LoRA).

Caveats

- **Vendor claims vs peer review:** CompactifAI's headline figures (93% memory, 80–95% "Slim" models, energy/cost numbers) come from Multiverse Computing's own arXiv preprint and press releases, not a peer-reviewed venue; the independent arXiv:2507.08836 study (SopraSteria) supports the efficiency/accuracy story but is itself a small evaluation. Treat the precise percentages as company-reported.
- **FLOPs ≠ latency:** Many reported "compression ratios" and "FLOPs reductions" do not translate to wall-clock speedups on real hardware; several TR/TT methods reduce parameters while *increasing* FLOPs. Always distinguish parameter compression, FLOPs reduction, and measured latency/energy.
- **Accuracy-recovery dependence:** Most strong results require fine-tuning/"healing." Reported accuracy drops assume this step; training-free numbers (pure post-training SVD) are typically worse at the same compression.
- **CP instability** is a recurring, well-documented problem (degeneracy, sensitivity to learning rate, infeasible memory for large layers) — the stable-CP and ADMM lines exist precisely because naive CP often fails.
- **Rank selection remains partly empirical:** VBMF, sensitivity, NAS, and Bayesian methods each have failure modes; no universal principled rank-selection method exists yet.
- **Recency/verification:** A few cited items are late-2025/2026 preprints (arXiv:2511.04494, 2511.03765, 2506.02818, and the PicoGPT MPO study arXiv:2603.28534) whose results are preliminary and not yet widely replicated. The PicoGPT MPO case study in particular is a ~1M-parameter toy model (reported up to 13× compression per block at bond dimension $\chi=4$, 97.7% of baseline token accuracy at $\chi=16$) — useful for pedagogy but not production evidence.

- **Date note:** This report is framed as of June 2026; some 2026-dated arXiv numbers may still be in revision.